tkinter-docs

E-Paine

May 21, 2021

CONTENTS

1 Tcl & Tk **3**

2 Widgets **5**
2.1 Commonly inherited widget methods . . . 5
2.2 BitmapImage . . . 10
2.3 Canvas . . . 10
2.4 PhotoImage . . . 35
2.5 Scrollbar . . . 35
2.6 Toplevel . . . 35

3 Tk Generic Properties **37**
3.1 Anchors . . . 37
3.2 Bitmaps and Stipples . . . 37
3.3 Colours . . . 37
3.4 Constants . . . 37
3.5 Cursors . . . 37
3.6 Dashes . . . 37
3.7 Events . . . 38
3.8 Fonts . . . 38
3.9 Getting Configuration Information . . . 39
3.10 Lengths . . . 39
3.11 Reliefs . . . 40

4 Glossary **41**

5 Tk license attribution **43**

Index **45**

This is designed as a reference for the regular Tk widgets. It is mostly just a 'translation' of the Tk 8.6 Man pages into what is hopefully a more understandable and newcomer-friendly form.

It is currently a work-in-progress (with lots of 'TODO's!).

CHAPTER

ONE

TCL & TK

```
class Tk
```

CHAPTER

TWO

WIDGETS

2.1 Commonly inherited widget methods

class Widget

A tkinter internal class inheriting common widget methods.

after(*ms*, *func=None*, **args*)

after_cancel(*id*)

after_idle(*func*, **args*)

anchor(*anchor=None*)

See `Widget.grid_anchor`.

bell(*displayof=0*)

bind(*sequence=None*, *func=None*, *add=None*)

bind_all(*sequence=None*, *func=None*, *add=None*)

bind_class(*className*, *sequence=None*, *func=None*, *add=None*)

bindtags(*tagList=None*)

cget(*key*)

clipboard_append(*string*, ***kw*)

clipboard_clear(***kw*)

clipboard_get(***kw*)

columnconfigure(*index*, *cnf={}*, ***kw*)

See `Widget.grid_columnconfigure`.

config(*cnf=None*, ***kw*)

See `Widget.configure`.

configure(*cnf=None*, ***kw*)

deletecommand(*name*)

destroy()

event_add(*virtual*, **sequences*)

event_delete(*virtual*, **sequences*)

event_generate(*sequence*, ***kw*)

event_info(*virtual=None*)

focus_displayof()

focus_force()

focus_get()

focus_lastfor()

focus_set()

forget()
See `Widget.pack_forget`.

getboolean(*s*)

getdouble(*s*)

getint(*s*)

getvar(*name='PY_VAR'*)

grab_current()

grab_release()

grab_set()

grab_set_global()

grab_status()

grid(*cnf={}*, ***kw*)
See `Widget.grid_configure`.

grid_anchor(*anchor=None*)

grid_bbox(*column=None*, *row=None*, *col2=None*, *row2=None*)

grid_configure(*cnf={}*, ***kw*)

grid_columnconfigure(*index*, *cnf={}*, ***kw*)

grid_forget()

grid_info()

grid_location(*x*, *y*)

grid_propagate(*flag=['_noarg_']*)

grid_remove()

grid_rowconfigure(*index*, *cnf={}*, ***kw*)

grid_size()

grid_slaves(*row=None*, *column=None*)

info()
See `Widget.pack_info`.

image_names()

image_types()

keys()

location(*x*, *y*)
See `Widget.grid_location`.

mainloop(*n=0*)

nametowidget(*name*)

option_add(*pattern*, *value*, *priority=None*)

option_clear()

option_get(*name*, *className*)

option_readfile(*fileName*, *priority=None*)

pack(*cnf={}*, ***kw*)
See `Widget.pack_configure`.

pack_configure(*cnf={}*, ***kw*)

pack_forget()

pack_info()

pack_propagate(*flag=['_noarg_']*)

pack_slaves()

place(*cnf={}*, ***kw*)
See `Widget.place_configure`.

place_configure(*cnf={}*, ***kw*)

place_forget()

place_info()

place_slaves()

propagate(*flag=['_noarg_']*)
See `Widget.pack_propagate`.

quit()

register(*func*, *subst=None*, *needcleanup=1*)

rowconfigure(*index*, *cnf={}*, ***kw*)
See `Widget.grid_rowconfigure`.

selection_clear(***kw*)

selection_get(***kw*)

selection_handle(*command*, ***kw*)

selection_own(***kw*)

selection_own_get(***kw*)

send(*interp*, *cmd*, **args*)

setvar(*name='PY_VAR'*, *value='1'*)

size()
See `Widget.grid_size`.

slaves()
See `Widget.pack_slaves`.

tk_bisque()

tk_focusFollowsMouse()

tk_focusNext()

tk_focusPrev()

tk_setPalette(**args*, ***kw*)

tk_strictMotif(*boolean=None*)

unbind(*sequence*, *funcid=None*)

unbind_all(*sequence*)

unbind_class(*className*, *sequence*)

update()

update_idletasks()

wait_variable(*name='PY_VAR'*)

wait_visibility(*window=None*)

wait_window(*window=None*)

waitvar(*name='PY_VAR'*)
See `Widget.wait_variable`.

winfo_atom(*name*, *displayof=0*)

winfo_atomname(*id*, *displayof=0*)

winfo_cells()

winfo_children()

winfo_class()

winfo_colormapfull()

winfo_containing(*rootX*, *rootY*, *displayof=0*)

winfo_depth()

winfo_exists()

winfo_fpixels(*number*)

winfo_geometry()

winfo_height()

winfo_id()

winfo_interps(*displayof=0*)

winfo_ismapped()

winfo_manager()

winfo_name()

winfo_parent()

winfo_pathname(*id*, *displayof=0*)

winfo_pixels(*number*)

winfo_pointerx()

winfo_pointerxy()

winfo_pointery()

winfo_reqheight()

winfo_reqwidth()

winfo_rgb(*color*)

winfo_rootx()

winfo_rooty()

winfo_screen()

winfo_screencells()

winfo_screendepth()

winfo_screenheight()

winfo_screenmmheight()

winfo_screenmmwidth()

winfo_screenvisual()

winfo_screenwidth()

winfo_server()

winfo_toplevel()

winfo_viewable()

winfo_visual()

winfo_visualid()

winfo_visualsavailable(*includeids=False*)

winfo_vrootheight()

winfo_vrootwidth()

winfo_vrootx()

winfo_vrooty()

winfo_width()

winfo_x()

winfo_y()

class XView

xview(**args*)

xview_moveto(*fraction*)

xview_scroll(*number*, *what*)

class YView

yview(**args*)

yview_moveto(*fraction*)

yview_scroll(*number*, *what*)

2.2 BitmapImage

class BitmapImage

2.3 Canvas

class Canvas(*master=None*, *cnf={}*, ***kw*)

Create a canvas widget for drawing graphics. It inherits all the common widget methods of `Widget`, `XView` and `YView`.

master is the parent widget of this canvas. If `None`, tkinter will attempt to use the *default root*.

cnf and *kw* are both used to specify widget options (see below). For example, `Canvas(root, {"bg": "white"})` and `Canvas(root, bg="white")` are equivalent.

Widget options:

background=

bg=

Specifies the normal background colour to use when displaying the widget. The given value may be any valid *Tk colour*. The default value is the platform default background colour.

borderwidth=

bd=

Specifies a non-negative value indicating the width of the 3-D border to draw around the outside of the widget (if such a border is being drawn; the *relief* option typically determines this). The value may also be used when drawing 3-D effects in the interior of the widget. The value may be any valid *Tk length*. The default value is 0.

closeenough=

Specifies a floating-point value indicating how close the mouse cursor must be to an item before it is considered to be "inside" the item. Defaults to 1.0.

confine=

Specifies a boolean value that indicates whether or not it should be allowable to set the canvas's view outside the region defined by the scrollRegion argument. If the value is `True`, the view will be constrained within the scroll region. The default value is `True`.

cursor=

Specifies the mouse cursor to be used for the widget. The value may be any valid *Tk cursor*. If an empty string is given, the widget should defer to its parent for cursor specification. The default value is to defer.

height=

Specifies a desired height that the canvas widget should request from its geometry manager. The value may be in any of the forms described in the *coordinates* section below. The default value is "7c".

highlightbackground=

Specifies the colour to use as a border highlight when the widget does not have focus. The given value may be any valid *Tk colour*. The default value is the platform default background colour.

highlightcolor=

Specifies the colour to use as a border highlight when the widget has focus. The given value may be any valid *Tk colour*. The default values are as follows:

MacOS	platform default foreground colour
Unix	black
Windows	platform default 'frame' colour

highlightthickness=

Specifies a non-negative value indicating the width of the highlight rectangle to draw around the outside of the widget when it has focus. The value may be any valid *Tk length*. If the value is zero, no focus highlight is drawn around the widget. The default values are as follows:

MacOS	3
Unix	1
Windows	2

insertbackground=

Specifies the colour to use as background in the area covered by the insertion cursor. The given value may be any valid *Tk colour*. This colour will normally override either the normal background for the widget (or the selection background if the insertion cursor happens to fall in the selection). The default values are as follows:

MacOS	black
Unix	black
Windows	platform default foreground colour

insertborderwidth=

Specifies a non-negative value indicating the width of the 3-D border to draw around the insertion cursor. The value may be any valid *Tk length*. The default value is 0.

insertontime=

Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain "off" in each blink cycle. If this option is zero then the cursor does not blink: it is on all the time. The default value is 300.

insertontime=

Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain "on" in each blink cycle. The default value is 600.

insertwidth=

Specifies a value indicating the total width of the insertion cursor. The value may be any valid *Tk length*. If a border has been specified for the insertion cursor (using the *insertborderwidth* option), the border will be drawn inside the width specified by the *insertwidth* option. The default value is 2.

relief=

Specifies the 3-D effect desired for the widget. The value may be any valid *Tk relief*. The value indicates how the interior of the widget should appear relative to its exterior; for example, raised means the interior of the widget should appear to protrude from the screen, relative to the exterior of the widget. The default value is "flat".

scrollregion=

Specifies a list with four coordinates describing the left, top, right and bottom coordinates of a rectangular region. This region is used for scrolling purposes and is considered to be the boundary of the information in the canvas. Each of the coordinates may be in any of the forms described in the *coordinates* section below. An empty string will make the scrollregion match the width and height of the canvas. The default value is an empty string.

selectbackground=

Specifies the background colour to use when displaying selected items. The given value may be any valid *Tk colour*. The default values are as follows:

MacOS	platform default selection colour
Unix	#c3c3c3 (light grey)
Windows	platform default highlight colour

selectborderwidth=

Specifies a non-negative value indicating the width of the 3-D border to draw around selected items. The value may be any valid *Tk length*. The default value is 0.

selectforeground=

Specifies the foreground colour to use when displaying selected items. The given value may be any valid *Tk colour*. The default values are as follows:

MacOS	platform default selection text colour
Unix	black
Windows	platform default highlight text colour

state=

Modifies the default state of the canvas where state may be set to one of: **normal**, **disabled**, or **hidden**. Individual canvas objects all have their own state option which may override the default state. Many options can take separate specifications such that the appearance of the item can be different in different situations. The options that start with "active" control the appearance when the mouse pointer is over it, while the options

starting with "disabled" control the appearance when the state is disabled. Canvas items which are disabled will not react to canvas bindings. The default value is **normal**.

takefocus=

Determines whether the window accepts the focus during keyboard traversal (e.g., Tab and Shift-Tab). Before setting the focus to a window, the traversal scripts consult the value of the *takefocus* option. A value of `False` means that the window should be skipped entirely during keyboard traversal. `True` means that the window should receive the input focus as long as it is viewable (it and all of its ancestors are mapped). An empty string for the option means that the traversal scripts make the decision about whether or not to focus on the window: the current algorithm is to skip the window if it is disabled, if it has no key bindings, or if it is not viewable.

If any other value is given, then the traversal scripts take the value, append the name of the window to it (with a separator space), and evaluate the resulting string as a Tcl script. The script must return 0, 1, or an empty string: a 0 or 1 value specifies whether the window will receive the input focus, and an empty string results in the default decision described above. The default value is an empty string.

Note: This interpretation of the option is defined entirely by the Tcl scripts that implement traversal: the widget implementations ignore the option entirely, so you can change its meaning if you redefine the keyboard traversal scripts.

---

width=

Specifies a desired width that the canvas widget should request from its geometry manager. The value may be in any of the forms described in the *coordinates* section below. The default value is "10c".

xscrollcommand=

Specifies the command used to communicate with horizontal scrollbars. When the view in the widget's window changes (or whenever anything else occurs that could change the display in a scrollbar, such as a change in the total size of the widget's contents), the widget will call the function with two numbers as arguments.

Each of the numbers is a fraction between 0 and 1, which indicates a position in the document. 0 indicates the beginning of the document, 1 indicates the end, .333 indicates a position one third the way through the document, and so on. The first number indicates the first information in the document that is visible in the window, and the second number indicates the information just after the last portion that is visible.

Typically, the *xscrollcommand* will be set to `Scrollbar.set`: this will cause the scrollbar to be updated whenever the view in the window changes. If this option is not specified, then no command will be executed. The default value is to call no function.

Note: Tkinter does not convert these numbers to Python floats for you, and instead leaves them as strings.

---

xscrollincrement=

Specifies an increment for horizontal scrolling, in the form of any valid *Tk length*. If the value of this option is greater than zero, the horizontal view in the window will be constrained so that the canvas x coordinate at the left edge of the window is always an even multiple of *xscrollincrement*; furthermore, the units for scrolling (e.g., the change in view when the left and right arrows of a scrollbar are selected) will also be *xscrollincrement*. If the value of this option is less than or equal to zero, then horizontal scrolling is unconstrained. The default value is 0.

yscrollcommand=

Specifies the prefix for a command used to communicate with vertical scrollbars. This option is treated in the same way as the *xscrollcommand* option, except that it is used for vertical scrollbars and is provided by widgets that support vertical scrolling. See the description of *xscrollcommand* for details on how this option is used. The default value is to call no function.

yscrollincrement=

Specifies an increment for vertical scrolling, in the form of any valid *Tk length*. If the value of this option is greater than zero, the vertical view in the window will be constrained so that the canvas y coordinate at the top edge of the window is always an even multiple of *yscrollincrement*; furthermore, the units for scrolling (e.g., the change in view when the top and bottom arrows of a scrollbar are selected) will also be *yscrollincrement*. If the value of this option is less than or equal to zero, then vertical scrolling is unconstrained. The default value is 0.

addtag_above(*newtag*, *tagOrId*)

For a single item just after (above) the one given by *tagOrId* in the display list, add *newtag* to the list of tags associated with the item if it is not already present on that list. It is possible that no items will be above the item given by *tagOrId*, in which case the command has no effect. If *tagOrId* denotes more than one item, then the last (topmost) of these items in the display list is used.

addtag_all(*newtag*)

For every item, add *newtag* to the list of tags associated with the item if it is not already present on that list. It is possible that there are no items, in which case the command has no effect.

addtag_below(*newtag*, *tagOrId*)

For a single item just before (below) the one given by *tagOrId* in the display list, add *newtag* to the list of tags associated with the item if it is not already present on that list. It is possible that no items will be below the item given by *tagOrId*, in which case the command has no effect. If *tagOrId* denotes more than one item, then the first (lowest) of these items in the display list is used.

addtag_closest(*newtag*, *x*, *y*, *halo=None*, *start=None*)

For the item closest to the point given by x and y, add *newtag* to the list of tags associated with the item if it is not already present on that list. It is possible that there are no items, in which case the command has no effect. If more than one item is at the same closest distance (e.g. two items overlap the point), then the top-most of these items (the last one in the display list) will have the new tag applied.

If *halo* is specified, then it must be a non-negative *length*. Any item closer than *halo* to the point is considered to overlap it. All items overlapping the halo are treated as if they have a distance of 0 from the given point.

If *start* is specified, it names an item using a tag or id (if by tag, it selects the bottom / first item in the display list with the given tag). Instead of adding *newtag* to the topmost closest item, this will tag the topmost closest item that is below *start* in the display list; if no such item exists, then it will behave as if the *start* argument had not been specified. This will only have an effect if *halo* is given.

addtag_enclosed(*newtag*, *x1*, *y1*, *x2*, *y2*)
For each item completely enclosed within the rectangular region given by *x1*, *y1*, *x2*, and *y2*, add *newtag* to the list of tags associated with the item if it is not already present on that list. It is possible that no items lie fully in this region, in which case the command has no effect. `(x1, y1)` must be the top-left corner of the region and `(x2, y2)` the bottom-right.

addtag_overlapping(*newtag*, *x1*, *y1*, *x2*, *y2*)
For each item overlapping the rectangular region given by *x1*, *y1*, *x2*, and *y2*, add *newtag* to the list of tags associated with the item if it is not already present on that list. It is possible that no items overlap this region, in which case the command has no effect. `(x1, y1)` must be the top-left corner of the region and `(x2, y2)` the bottom-right.

addtag_withtag(*newtag*, *tagOrId*)
For each item specified by *tagOrId*, add *newtag* to the list of tags associated with the item if it is not already present on that list. It is possible that no items have this tag / id, in which case the command has no effect.

bbox(**args*)
Returns a tuple with four integers giving an approximate bounding box for all the items given in *args*. The tuple has the form `(x1, y1, x2, y2)`, such that the drawn areas of all the given elements are within the region bounded by x1 on the left, x2 on the right, y1 on the top, and y2 on the bottom. The return value may overestimate the actual bounding box by a few pixels. If no items match any of the item given in *args* or if the matching items have empty bounding boxes (i.e. they have nothing to display) then `None` is returned.

canvasx(*self*, *screenx*, *gridspacing=None*)
Given a window x-coordinate in the canvas *screenx*, this command returns the canvas x-coordinate that is displayed at that location. If *gridspacing* is specified, then the canvas coordinate is rounded to the nearest multiple of *gridspacing* units.

canvasy(*screeny*, *gridspacing=None*)
Given a window y-coordinate in the canvas *screeny*, this command returns the canvas y-coordinate that is displayed at that location. If *gridspacing* is specified, then the canvas coordinate is rounded to the nearest multiple of *gridspacing* units.

coords(**args*)
Query or modify the coordinates that define an item. The first argument should always be the tag / id of a canvas item. If no coordinates are specified (i.e. the only argument given is the item tag / id), this command returns a tuple whose elements are the coordinates of the item.

If coordinates are specified, then they replace the current coordinates for the given item. If the tag / id refers to multiple items, then the bottom / first one in the display list is used.

Note: For rectangles, ovals and arcs the returned list of coordinates has a fixed order, namely the left, top, right and bottom coordinates, which may not be the order originally given. Also the coordinates are always returned in screen units with no units (that is, in pixels). So if the original coordinates were specified for instance in centimeters or inches, the returned values will nevertheless be in pixels.

---

create_arc(**args*, ***kw*)
Draw an arc, chord or pieslice. Returns the item id.

args is two coordinate points specifying a rectangle containing the oval (from which part is taken to draw the arc). Because tkinter flattens these, both `(x1, y1, x2, y2)` and `((x1, y1), (x2, y2))` are acceptable.

kw is the options, which can be any of the following:

dash=
activedash=
disableddash=
These options specifies dash patterns for the normal, active and disabled states of the outline of the arc (correspondingly). The value may be any valid *Tk dash style*. The default value is a solid outline.

dashoffset=
The starting offset in pixels into the pattern provided by the *dash* option. *dashoffset* is ignored if there is no *dash* pattern. The offset may have any of the forms described in the *coordinates* section below. The default value is 0.

extent=
Specifies the size of the angular range occupied by the arc. The arc's range extends for the given number of degrees counter-clockwise from the starting angle given by the *start* option. The value may be negative. If it is greater than 360 or less than -360, then degrees modulo 360 is used as the extent. The default value is 90.

fill=
activefill=
disabledfill=
Specifies the colour to be used to fill arc's area in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If the value is an empty string, then the arc will not be filled (i.e. it will be transparent). The default value is an empty string.

offset=
Specifies the offset of stipples. The offset value can be of the form `"x,y"` or side, where side can be **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. In the first case, the origin is the origin of the canvas itself, but putting # in front of the coordinate pair indicates using the current window's origin instead. The default value is `"0,0"`.

Warning: Stipple offsets are only supported on Unix; they are silently ignored on other platforms.

Note: A Python tuple of `(x, y)` cannot be given and instead must be manually formatted to string of the correct format (`"x,y"` or `"#x,y"`).

outline=
activeoutline=
disabledoutline=
These options specifies the colour that should be used to draw the outline of the arc in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If colour is specified as an empty string then no outline is drawn for the arc. The default values are as follows:

MacOS	platform default foreground colour
Unix	black
Windows	platform default foreground colour

outlineoffset=
Specifies the offset of the stipple pattern used for outlines, in the same way that the *offset* option controls fill stipples. See the *offset* option for a description of acceptable values. The default value is `"0,0"`.

outlinestipple=
activeoutlinestipple=
disabledoutlinestipple=
This option specifies stipple patterns that should be used to draw the outline of the arc in its normal, active and disabled states (correspondingly). It indicates that the outline for the arc should be drawn with a stipple pattern and specifies the stipple pattern to use. The given value may be any valid *Tk Bitmap*. If the *outline* option has not been specified then this option has no effect. If the value is an empty string, then the outline is drawn in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

stipple=
activestipple=
disabledstipple=
This option specifies stipple patterns that should be used to fill the arc in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk Bitmap*. If the *fill* option has not been specified then this option has no effect. If the value is an empty string, then filling is done in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

start=
Specifies the beginning of the angular range occupied by the arc. The value is given in degrees measured counter-clockwise from the 3-o'clock position; it may be either positive or negative. The default value is 0.

state=
This allows the arc to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

style=

Specifies how to draw the arc. If type is **pieslice** then the arc's region is defined by a section of the oval's perimeter plus two lines between the center of the oval and each end of the perimeter section. If type is **chord** then the arc's region is defined by a section of the oval's perimeter plus a single line connecting the two end points of the perimeter section. If type is **arc** then the arc's region consists of a section of the perimeter alone. In this last case the *fill* option is ignored. The default value is **pieslice**.

tags=
Specifies one or more tags to apply to the arc. When used in `Canvas.itemconfigure`, this replaces any existing tags for the arc. An empty list may also be specified. The default value is an empty list.

width=
activewidth=
disabledwidth=
Specifies the width of the outline to be drawn around the arc's region, in its normal, active and disabled states (correspondingly). The value may be in any of the forms described in the *coordinates* section below. If the *outline* option has been specified as an empty string, then this option has no effect. The default value is 1.

Note: Wide outlines will be drawn centered on the edges of the arc's region.

`create_bitmap`(**args*, ***kw*)
Draw a bitmap. Returns the item id.

args is a single coordinate point `(x, y)`.

kw is the options, which can be any of the following:

anchor=
The given value determines how to position the bitmap relative to the positioning coordinate; it may have any valid *Tk anchor*. For example, if the value is **center** then the bitmap is centered on the point; if the value is **n** then the bitmap will be drawn so that its top center point is at the positioning coordinate. The default value is **center**.

background=
activebackground=
disabledbackground=
Specifies the colour to use for each of the bitmap's "0" valued pixels in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If the value is an empty string, then nothing is displayed where the bitmap pixels are 0; this produces a transparent effect. The default value is an empty string.

bitmap=
activebitmap=
disabledbitmap=
Specifies the bitmap to display in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk Bitmap*. An empty string specifies no bitmap. The default value is an empty string.

foreground=
activeforeground=
disabledforeground=
Specifies the colour to use for each of the bitmap's "1" valued pixels in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*.

state=
This allows the bitmap to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=
Specifies one or more tags to apply to the bitmap. When used in `Canvas.itemconfigure`, this replaces any existing tags for the bitmap. An empty list may also be specified. The default value is an empty list.

create_image(**args*, ***kw*)
Draw an image. Returns the item id.

args is a single coordinate point `(x, y)`.

kw is the options, which can be any of the following:

anchor=
The given value determines how to position the image relative to the positioning coordinate; it may have any valid *Tk anchor*. For example, if the value is **center** then the image is centered on the point; if the value is **n** then the image will be drawn so that its top center point is at the positioning coordinate. The default value is **center**.

image=
activeimage=
disabledimage=
Specifies the image to display in the item in is normal, active and disabled states (correspondingly). The image must be a `BitmapImage`, `PhotoImage` or similar.

state=
This allows the image to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=
Specifies one or more tags to apply to the image. When used in `Canvas.itemconfigure`, this replaces any existing tags for the image. An empty list may also be specified. The default value is an empty list.

create_line(**args*, ***kw*)
Draw a line. Returns the item id.

args is two or more coordinate points of the line. Because tkinter flattens these, both `(x1, y1, ..., xn, yn)` and `((x1, y1), ..., (xn, yn))` are acceptable.

kw is the options, which can be any of the following:

arrow=

Indicates whether or not arrowheads are to be drawn at one or both ends of the line. The value must have one of the values **none** (for no arrowheads), **first** (for an arrowhead at the first point of the line), **last** (for an arrowhead at the last point of the line), or **both** (for arrowheads at both ends). When requested to draw an arrowhead, Tk internally adjusts the corresponding line end point so that the rendered line ends at the neck of the arrowhead rather than at its tip so that the line doesn't extend past the edge of the arrowhead. This may trigger a **Leave** event if the mouse is hovering this line end (see the *events* section). Conversely, when removing an arrowhead Tk adjusts the corresponding line point the other way round, which may trigger an **Enter** event. The default value is **none**.

arrowshape=

This option indicates how to draw arrowheads. The shape argument must be a tuple / list with three elements, each specifying a distance in any of the forms described in the *coordinates* section below. The first element of the list gives the distance along the line from the neck of the arrowhead to its tip (**l1** in the diagram). The second element gives the distance along the line from the trailing points of the arrowhead to the tip (**l2**), and the third element gives the distance from the outside edge of the line to the trailing points (**l3**). The default value is `(8, 10, 3)`.

capstyle=

Specifies the ways in which caps are to be drawn at the endpoints of the line. The value may be any of **butt**, **projecting**, or **round**. Where arrowheads are drawn, the cap style is ignored. The default value is **butt**.

dash=

activedash=

disableddash=

These options specifies dash patterns for the normal, active and disabled states of the line (correspondingly). The value may be any valid *Tk dash style*. The default value is a solid line.

dashoffset=

The starting offset in pixels into the pattern provided by the *dash* option. *dashoffset* is ignored if there is no *dash* pattern. The offset may have any of the forms described in the *coordinates* section below. The default value is 0.

fill=

activefill=

disabledfill=

Specifies the colour used to draw the line in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If the value is an empty string, then the line will not be filled (i.e.

it will be transparent). The default values are as follows:

MacOS	platform default foreground colour
Unix	black
Windows	platform default foreground colour

joinstyle=
Specifies the ways in which joints are to be drawn at the vertices of the line (only applicable if more than 2 coordinates are given). The value may be any of **bevel**, **miter**, or **round**. The default value is **round**.

smooth=
This value must either be a boolean or a line smoothing method.

`True`
bezier
The line should be drawn as a curve, rendered as a set of quadratic splines: one spline is drawn for the first and second line segments, one for the second and third, and so on. Straight-line segments can be generated within a curve by duplicating the end-points of the desired line segment.

raw
The line should also be drawn as a curve but where the list of coordinates is such that the first coordinate pair (and every third coordinate pair thereafter) is a knot point on a cubic Bezier curve, and the other coordinates are control points on the cubic Bezier curve. Straight line segments can be generated within a curve by making control points equal to their neighbouring knot points. If the last point is a control point and not a knot point, the point is repeated (one or two times) so that it also becomes a knot point.

`False`
empty string
No smoothing is applied.

The default value is `False`.

splinesteps=
Specifies the degree of smoothness desired for curves: each spline will be approximated with number line segments. This option is ignored if the *smooth* option is `False` or an empty string. The default value is 12.

stipple=
activestipple=
disabledstipple=
This option specifies stipple patterns that should be used to fill the line in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk Bitmap*. If the *fill* option is an empty string,

then this option has no effect. If the value is an empty string, then filling is done in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

state=

This allows the line to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=

Specifies one or more tags to apply to the line. When used in `Canvas.itemconfigure`, this replaces any existing tags for the line. An empty list may also be specified. The default value is an empty list.

width=

activewidth=

disabledwidth=

Specifies the width the line to be drawn, in its normal, active and disabled states (correspondingly). The value may be in any of the forms described in the *coordinates* section below. If the *fill* option has been specified as an empty string, then this option has no effect. The default value is 1.

create_oval(**args*, ***kw*)

Draw an oval. Returns the item id.

args is two coordinate points specifying a rectangle containing the oval. Because tkinter flattens these, both `(x1, y1, x2, y2)` and `((x1, y1), (x2, y2))` are acceptable.

kw is the options, which can be any of the following:

dash=

activedash=

disableddash=

These options specifies dash patterns for the normal, active and disabled states of the outline of the oval (correspondingly). The value may be any valid *Tk dash style*. The default value is a solid line.

dashoffset=

The starting offset in pixels into the pattern provided by the *dash* option. *dashoffset* is ignored if there is no *dash* pattern. The offset may have any of the forms described in the *coordinates* section below. The default value is 0.

fill=

activefill=

disabledfill=

Specifies the colour to be used to fill oval's area in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If the value is an empty string, then the oval will not be filled (i.e. it will be transparent). The default value is an empty string.

offset=

Specifies the offset of stipples. The offset value can be of the form `"x,y"` or side, where side can be **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. In the first case, the origin is the origin of the canvas itself, but putting # in front of the coordinate pair indicates using the current window's origin instead. The default value is `"0,0"`.

> **Warning:** Stipple offsets are only supported on Unix; they are silently ignored on other platforms.

Note: A Python tuple of `(x, y)` cannot be given and instead must be manually formatted to string of the correct format (`"x,y"` or `"#x,y"`).

---

outline=

activeoutline=

disabledoutline=

These options specifies the colour that should be used to draw the outline of the oval in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If colour is specified as an empty string then no outline is drawn for the oval. The default values are as follows:

MacOS	platform default foreground colour
Unix	black
Windows	platform default foreground colour

outlineoffset=

Specifies the offset of the stipple pattern used for outlines, in the same way that the *offset* option controls fill stipples. See the *offset* option for a description of acceptable values. The default value is `"0,0"`.

outlinestipple=

activeoutlinestipple=

disabledoutlinestipple=

This option specifies stipple patterns that should be used to draw the outline of the oval in its normal, active and disabled states (correspondingly). It indicates that the outline for the oval should be drawn with a stipple pattern and specifies the stipple pattern to use. The given value may be any valid *Tk Bitmap*. If the *outline* option has not been specified then this option has no effect. If the value is an empty string, then the outline is drawn in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

stipple=
activestipple=
disabledstipple=
This option specifies stipple patterns that should be used to fill the oval in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk Bitmap*. If the *fill* option has not been specified then this option has no effect. If the value is an empty string, then filling is done in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

state=
This allows the oval to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=
Specifies one or more tags to apply to the oval. When used in `Canvas.itemconfigure`, this replaces any existing tags for the oval. An empty list may also be specified. The default value is an empty list.

width=
activewidth=
disabledwidth=
Specifies the width of the outline to be drawn around the oval's region, in its normal, active and disabled states (correspondingly). The value may be in any of the forms described in the *coordinates* section below. If the *outline* option has been specified as an empty string, then this option has no effect. The default value is 1.

create_polygon(**args*, ***kw*)
Draw a polygon. Returns the item id.

args is two or more coordinate points of the polygon. These will be it's vertices (corners). Because tkinter flattens these, both `(x1, y1, ..., xn, yn)` and `((x1, y1), ..., (xn, yn))` are acceptable.

kw is the options, which can be any of the following:

dash=
activedash=
disabledash=
These options specifies dash patterns for the normal, active and disabled states of the outline of the polygon (correspondingly). The value may be any valid *Tk dash style*. The default value is a solid line.

dashoffset=
The starting offset in pixels into the pattern provided by the *dash* option. *dashoffset* is ignored if there is no *dash* pattern. The offset may have any of the forms described in the *coordinates* section below. The default value is 0.

fill=
activefill=
disabledfill=
Specifies the colour to be used to fill polygon's area in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If the value is an empty string, then the oval will not be filled (i.e. it will be transparent). The default value is an empty string.

joinstyle=
Specifies the ways in which joints are to be drawn at the vertices of the polygon. The value may be any of **bevel**, **miter**, or **round**. The default value is **round**.

offset=
Specifies the offset of stipples. The offset value can be of the form `"x,y"` or side, where side can be **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. In the first case, the origin is the origin of the canvas itself, but putting # in front of the coordinate pair indicates using the current window's origin instead. The default value is `"0,0"`.

Warning: Stipple offsets are only supported on Unix; they are silently ignored on other platforms.

Note: A Python tuple of `(x, y)` cannot be given and instead must be manually formatted to string of the correct format (`"x,y"` or `"#x,y"`).

outline=
activeoutline=
disabledoutline=
These options specifies the colour that should be used to draw the outline of the oval in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If colour is specified as an empty string then no outline is drawn for the oval. The default values are as follows:

MacOS	platform default foreground colour
Unix	black
Windows	platform default foreground colour

outlineoffset=
Specifies the offset of the stipple pattern used for outlines, in the same way that the *offset* option controls fill stipples. See the *offset* option for a description of acceptable values. The default value is `"0,0"`.

outlinestipple=
activeoutlinestipple=
disabledoutlinestipple=
This option specifies stipple patterns that should be used to draw the outline of the oval in its normal, active and disabled states (correspondingly). It indicates that the outline for the oval should be drawn with a stipple pattern and specifies the stipple pattern to use. The given value may be any valid *Tk Bitmap*. If the *outline* option has not been specified then this option has no effect. If the value is an empty string, then the outline is drawn in a solid fashion. The default value is an empty string.

smooth=
This value must either be a boolean or a line smoothing method.

`True`
bezier
The outline should be drawn as a curve, rendered as a set of quadratic splines: one spline is drawn for the first and second line segments, one for the second and third, and so on. Straight-line segments can be generated within a curve by duplicating the end-points of the desired line segment.

raw
The outline should also be drawn as a curve but where the list of coordinates is such that the first coordinate pair (and every third coordinate pair thereafter) is a knot point on a cubic Bezier curve, and the other coordinates are control points on the cubic Bezier curve. Straight line segments can be generated within a curve by making control points equal to their neighbouring knot points. If the last point is a control point and not a knot point, the point is repeated (one or two times) so that it also becomes a knot point.

`False`
empty string
No smoothing is applied.

The default value is `False`.

splinesteps=
Specifies the degree of smoothness desired for curves: each spline will be approximated with number line segments. This option is ignored if the *smooth* option is `False` or an empty string. The default value is 12.

stipple=
activestipple=
disabledstipple=
This option specifies stipple patterns that should be used to fill the polygon in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk Bitmap*. If the *fill* option has not been specified then this option has no effect. If the value is an empty string, then filling is done in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

state=
This allows the polygon to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=
Specifies one or more tags to apply to the polygon. When used in `Canvas.itemconfigure`, this replaces any existing tags for the polygon. An empty list may also be specified. The default value is an empty list.

width=
activewidth=
disabledwidth=
Specifies the width of the outline to be drawn around the polygon's region, in its normal, active and disabled states (correspondingly). The value may be in any of the forms described in the *coordinates* section below. If the *outline* option has been specified as an empty string, then this option has no effect. The default value is 1.

create_rectangle(**args*, ***kw*)
Draw a rectangle. Returns the item id.

args is two coordinate points specifying opposite corners of the rectangle. Because tkinter flattens these, both `(x1, y1, x2, y2)` and `((x1, y1), (x2, y2))` are acceptable.

kw is the options, which can be any of the following:

dash=
activedash=
disableddash=
These options specifies dash patterns for the normal, active and disabled states of the outline of the rectangle (correspondingly). The value may be any valid *Tk dash style*. The default value is a solid line.

dashoffset=
The starting offset in pixels into the pattern provided by the *dash* option. *dashoffset* is ignored if there is no *dash* pattern. The offset may have any of the forms described in the *coordinates* section below. The default value is 0.

fill=
activefill=
disabledfill=
Specifies the colour to be used to fill rectangle's area in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If the value is an empty string, then the rectangle will not be filled (i.e. it will be transparent). The default value is an empty string.

offset=
Specifies the offset of stipples. The offset value can be of the form `"x,y"` or side, where side can be **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. In the first case, the origin is the origin of the canvas itself, but putting # in front of the coordinate pair indicates using the current window's origin instead. The default value is `"0,0"`.

> **Warning:** Stipple offsets are only supported on Unix; they are silently ignored on other platforms.

Note: A Python tuple of `(x, y)` cannot be given and instead must be manually formatted to string of the correct format (`"x,y"` or `"#x,y"`).

---

outline=
activeoutline=
disabledoutline=
These options specifies the colour that should be used to draw the outline of the rectangle in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If colour is specified as an empty string then no outline is drawn for the rectangle. The default values are as follows:

MacOS	platform default foreground colour
Unix	black
Windows	platform default foreground colour

outlineoffset=
Specifies the offset of the stipple pattern used for outlines, in the same way that the *offset* option controls fill stipples. See the *offset* option for a description of acceptable values. The default value is `"0,0"`.

outlinestipple=
activeoutlinestipple=
disabledoutlinestipple=
This option specifies stipple patterns that should be used to draw the outline of the rectangle in its normal, active and disabled states (correspondingly). It indicates that the outline for the rectangle should be drawn with a stipple pattern and specifies the stipple pattern to use. The given value may be any valid *Tk Bitmap*. If the *outline* option has not been specified then this option has no effect. If the value is an empty string, then the outline is drawn in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

stipple=
activestipple=

disabledstipple=

This option specifies stipple patterns that should be used to fill the rectangle in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk Bitmap*. If the *fill* option has not been specified then this option has no effect. If the value is an empty string, then filling is done in a solid fashion. The default value is an empty string.

> **Warning:** Stipples are not well supported on platforms other than Unix.

state=

This allows the rectangle to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=

Specifies one or more tags to apply to the rectangle. When used in `Canvas.itemconfigure`, this replaces any existing tags for the rectangle. An empty list may also be specified. The default value is an empty list.

width=

activewidth=

disabledwidth=

Specifies the width of the outline to be drawn around the rectangle's region, in its normal, active and disabled states (correspondingly). The value may be in any of the forms described in the *coordinates* section below. If the *outline* option has been specified as an empty string, then this option has no effect. The default value is 1.

create_text(**args*, ***kw*)

Draw text. Returns the item id.

args is a single coordinate point `(x, y)`.

kw is the options, which can be any of the following:

anchor=

The given value determines how to position the text relative to the positioning coordinate; it may have any valid *Tk anchor*. For example, if the value is **center** then the text is centered on the point; if the value is **n** then the text will be drawn so that its top center point is at the positioning coordinate. The default value is **center**.

angle=

This value is how many degrees to rotate the text anticlockwise about the positioning point for the text; it may have any floating-point value from 0.0 to 360.0. For example, if the value is 90, then the text will be drawn vertically from bottom to top. The default value is 0.

fill=
activefill=
disabledfill=
Specifies the colour used to draw the text in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk colour*. If the value is an empty string, then the text will not be filled (i.e. it will be transparent). The default values are as follows:

MacOS	platform default foreground colour
Unix	black
Windows	platform default foreground colour

font=
Specifies the font to use for the text item. The value may be any valid *Tk font*. The default value is **TkDefaultFont**.

justify=
Specifies how to justify the text within its bounding region. The value must be one of **left**, **right**, or **center**. This option will only matter if the text is displayed as multiple lines. The default value is **left**.

stipple=
activestipple=
disabledstipple=
This option specifies stipple patterns that should be used to fill the line in its normal, active and disabled states (correspondingly). The given value may be any valid *Tk Bitmap*. If the *fill* option is an empty string, then this option has no effect. If the value is an empty string, then filling is done in a solid fashion. The default value is an empty string.

Warning: Stipples are not well supported on platforms other than Unix.

state=
This allows the rectangle to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=
Specifies one or more tags to apply to the rectangle. When used in `Canvas.itemconfigure`, this replaces any existing tags for the rectangle. An empty list may also be specified. The default value is an empty list.

text=

String specifies the characters to be displayed in the text item. Newline characters cause line breaks. The characters in the item may also be changed with the `Canvas.dchars` and `Canvas.insert` methods. The default value is an empty string.

underline=

Specifies the integer index of a character within the text to be underlined. 0 corresponds to the first character of the text displayed, 1 to the next character, and so on. -1 means that no underline should be drawn. Multiple characters cannot be underlined with the exception of the whole text, where the appropriate font should be used instead. The default value is -1.

width=

Specifies a maximum line length for the text, in any of the forms described in the *coordinates* section below. If this option is 0 the text is broken into lines only at newline characters. However, if this option is non-zero then any line that would be longer than lineLength is broken just before a space character to make the line shorter than the given length; the space character is treated as if it were a newline character. The default value is 0.

`create_window`(**args*, ***kw*)

'Embeds' another widget on the canvas. Returns the item id.

args is a single coordinate point `(x, y)`.

kw is the options, which can be any of the following:

anchor=

The given value determines how to position the widget relative to the positioning coordinate; it may have any valid *Tk anchor*. For example, if the value is **center** then the widget is centered on the point; if the value is **n** then the window will be drawn so that its top center point is at the positioning coordinate. The default value is **center**.

height=

Specifies the height to assign to the item's widget. The value may have any of the forms described in the *coordinates* section below. If the value is specified as 0, then the window is given whatever height it requests internally. The default value is 0.

state=

This allows the rectangle to override the canvas widget's global *state* option. It takes the same values: **normal**, **disabled** or **hidden**. An empty string will defer to the canvas widget's state. The default value is an empty string.

tags=

Specifies one or more tags to apply to the rectangle. When used in `Canvas.itemconfigure`, this replaces any existing tags for the rectangle. An empty list may also be specified. The default value is an empty list.

width=

Specifies the width to assign to the item's widget. The value may have any of the forms described in the *coordinates* section below. If the value is specified as 0, then the window is given whatever height it requests internally. The default value is 0.

window=

Specifies the widget to associate with this item. The widget specified must either be a child of the canvas widget or a child of some ancestor of the canvas widget. The given widget must not be a `Toplevel` window.

dchars(**args*)

For each item given by the first value of *args* (item id or tag), delete the characters, or coordinates, in the range given by second and third values of *args*, inclusive. If some of the items given by do not support indexing operations then they ignore this operation. Text items interpret second and third values of *args* as indices to a character, line and polygon items interpret them as indices to a coordinate (an x,y pair). Indices are described in *indices* section below. If only two values are given by *args*, the third value defaults to be the same as the second.

delete(**args*)

Delete each of the items given by the first (and only) value of *args* (this should be an item id or tag).

dtag(**args*)

For each of the items given by the first value of *args* (this should be an item id or tag), delete the tag given by the second value of *args* from the list of those associated with the item. If an item does not have the tag specified for deletion, then the item is unaffected by the command. If only one value is given in *args*, it defaults to be the same as that of the first value.

find_above(*tagOrId*)

Returns a single item just after (above) the one given by *tagOrId* in the display list, regardless of current visibility. If *tagOrId* denotes more than one item, then the last (topmost) of these items in the display list is used.

find_all()

Returns every item on the canvas, regardless of current visibility. The items are returned in stacking order, with the lowest item first.

find_below(*tagOrId*)

Returns a single item just before (below) the one given by *tagOrId* in the display list, regardless of current visibility. If *tagOrId* denotes more than one item, then the first (lowest) of these items in the display list is used.

find_closest(*x*, *y*, *halo=None*, *start=None*)

Returns the closest visible item to the point given by x and y. If more than one item is at the same closest distance (e.g. two items overlap the point), then the top-most of these items (the last one in the display list) will be returned.

If *halo* is specified, then it must be a non-negative *length*. Any item closer than *halo* to the point is considered to overlap it. All items overlapping the halo are treated as if they have a distance of 0 from the given point.

If *start* is specified, it names an item using a tag or id (if by tag, it selects the bottom / first item in the display list with the given tag). Instead of returning the topmost closest item, this will return the topmost closest item that is below *start* in the display list; if no such item exists, then it will behave as if the *start* argument had not been specified. This will only have an effect if *halo* is given.

find_enclosed(*x1*, *y1*, *x2*, *y2*)

Returns every item completely enclosed within the rectangular region given by *x1*, *y1*, *x2*, and *y2*. `(x1,`

`y1)` must be the top-left corner of the region and `(x2, y2)` the bottom-right. The items are returned in stacking order, with the lowest item first.

find_overlapping(*x1*, *y1*, *x2*, *y2*)
Returns every item overlapping the rectangular region given by *x1*, *y1*, *x2*, and *y2*. `(x1, y1)` must be the top-left corner of the region and `(x2, y2)` the bottom-right. The items are returned in stacking order, with the lowest item first.

find_withtag(*tagOrId*)
Returns every item specified by *tagOrId*. The items are returned in stacking order, with the lowest item first. If an id is given, the a tuple of just that id will be returned and therefore most uses of this method will provide a tag instead.

focus(**args*)
Set the keyboard focus for the canvas widget to the item given by the first value of *args*. If this value refers to several items, then the focus is set to the first (lowest) such item in the display list that supports the insertion cursor. If tagOrId does not refer to any items, or if none of them support the insertion cursor, then the focus is not changed. If the first value of *args* is an empty string, then the focus item is reset so that no item has the focus. If *args* is empty then the command returns the id for the item that currently has the focus, or an empty string if no item has the focus.

Once the focus has been set to an item, the item will display the insertion cursor and all keyboard events will be directed to that item. The focus item within a canvas and the focus item on the window are totally independent: a given item does not actually have the input focus unless it satisfies both of the following:

(a) its canvas is the focus widget

(b) the item is the focus item within the canvas

In most cases, it is advisable to follow `Canvas.focus` with one of `Widget.focus_force` or `Widget.focus_set` to set the focus widget to the canvas (if it was not there already).

gettags(**args*)
This can be somewhat thought of as the inverse of `Canvas.find_withtag`; given a single value in *args*, this will return the tags for that item. If multiple items are specified by the given value, then the tags of the first (lowest) item in the display list are returned.

icursor(**args*)
Set the position of the insertion cursor for the item(s) given by the first value of *args* to just before the character whose position is given by the second value of *args*. If some or all of the items given by the first value do not support an insertion cursor then this command has no effect on them. See the *indices* section below for a description of the legal forms for index.

Note: The insertion cursor is only displayed in an item if that item currently has the keyboard focus (see the `Canvas.focus` method above), but the cursor position may be set even when the item does not have the focus.

---

index(**args*)
This command returns an integer giving the numerical index within the first value of *args* corresponding to the second value. The second value gives a textual description of the desired position as described in *indices* below. Text items interpret index as an index to a character while line and polygon items interpret it as an index to a coordinate (an x,y pair). The return value is guaranteed to lie between 0 and the number of characters or coordinates within the item (inclusive). If the first value of *args* refers to multiple items, then the index is processed in the first (lowest) of these items that supports indexing operations.

insert(**args*)
For each of the items given by the first value of *args*, if the item supports text or coordinate, insertion then the third value of *args* is inserted into the item's text just before the character, or coordinate, whose index

is that given by the second value of *args*. Text items interpret the second value as an index to a character while line and polygon items interpret it as an index to a coordinate (an x,y pair). For lines and polygons the third value of *args* must be a valid coordinate sequence. See *indices* below for information about the forms allowed for the second value of *args*.

itemcget(*tagOrId*, *option*)
Returns the current value of the configuration option for the item given by *tagOrId* whose name is *option*. This command is similar to the `Widget.cget` method except that it applies to a particular item rather than the widget as a whole. *option* may have any of the options accepted by the create_* method used to first create the item. If *tagOrId* is a tag that refers to more than one item, the first (lowest) such item is used.

itemconfig(*tagOrId*, *cnf=None*, ***kw*)
See `Canvas.itemconfigure`.

itemconfigure(*tagOrId*, *cnf=None*, ***kw*)
This command is similar to the `Widget.configure` method except that it modifies item-specific options for the items given by *tagOrId* instead of modifying options for the overall canvas widget. If no option/s are specified, this returns a list describing all of the available options for the first item given by *tagOrId* (see *getting configuration information* for information on the format of the return value). If one or more option-value pairs are specified, then the command modifies the given widget option/s to have the given value/s in each of the items given by *tagOrId*; in this case the method returns `None`. The options and values are the same as those accepted by the create_* method used to first create the item/s.

lift(**args*)
See `Canvas.tag_raise`.

lower(**args*)
See `Canvas.tag_lower`.

move(**args*)

moveto(*tagOrId*, *x=''*, *y=''*)

postscript(*cnf={}*, ***kw*)

scale(**args*)

scan_dragto(*x*, *y*, *gain=10*)

scan_mark(*x*, *y*)

select_adjust(*tagOrId*, *index*)

select_clear()

select_from(*tagOrId*, *index*)

select_item()

select_to(*tagOrId*, *index*)

tag_bind(*tagOrId*, *sequence=None*, *func=None*, *add=None*)

tag_lower(**args*)

tag_raise(**args*)

tag_unbind(*tagOrId*, *sequence*, *funcid=None*)

tkraise(**args*)
See `Canvas.tag_raise`.

type(*tagOrId*)

2.3.1 Coordinates

TODO

2.3.2 Indices

TODO

2.4 PhotoImage

class PhotoImage

2.5 Scrollbar

class Scrollbar

set()

2.6 Toplevel

class Toplevel

CHAPTER

THREE

TK GENERIC PROPERTIES

3.1 Anchors

TODO

3.2 Bitmaps and Stipples

TODO

3.3 Colours

TODO

3.4 Constants

Whether you use the constants or type the strings is completely down to personal preference.

3.5 Cursors

TODO

3.6 Dashes

TODO

3.7 Events

TODO

3.8 Fonts

TkDefaultFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	13	normal
Unix	Helvetica or sans-serif	10	normal
Windows	MS Sans Serif or Tahoma	8	normal

TkTextFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	13	normal
Unix	Helvetica or sans-serif	10	normal
Windows	MS Sans Serif or Tahoma	8	normal

TkHeadingFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	11	normal
Unix	Helvetica or sans-serif	10	bold
Windows	MS Sans Serif or Tahoma	8	normal

TkCaptionFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	13	bold
Unix	Helvetica or sans-serif	12	bold
Windows	MS Sans Serif or Tahoma	8	bold

TkTooltipFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	11	normal
Unix	Helvetica or sans-serif	9	normal
Windows	MS Sans Serif or Tahoma	8	normal

TkFixedFont

Platform	Font Family	Font Size	Font Weight
MacOS	Monaco	11	normal
Unix	courier or monospace	10	normal
Windows	Courier	10	normal

TkIconFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	13	normal
Unix	Helvetica or sans-serif	10	normal
Windows	MS Sans Serif or Tahoma	8	normal

TkMenuFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	14	normal
Unix	Helvetica or sans-serif	10	normal
Windows	MS Sans Serif or Tahoma	8	normal

TkSmallCaptionFont

Platform	Font Family	Font Size	Font Weight
MacOS	Lucida Grande	10	normal
Unix	Helvetica or sans-serif	9	normal
Windows	MS Sans Serif or Tahoma	8	normal

3.9 Getting Configuration Information

3.10 Lengths

A valid length may have any of the following forms (where # is a floating-point number):

#
The number specifies a distance in pixels.

#c
The number specifies a distance in centimeters on the screen.

#i
The number specifies a distance in inches on the screen.

#m
The number specifies a distance in millimeters on the screen.

#p
The number specifies a distance in printer's points (1/72 inch) on the screen.

Whitespace is permitted between the number and distance-unit character.

3.11 Reliefs

TODO

CHAPTER

FOUR

GLOSSARY

default root The default root is an instance of the *Tk* class, cached by tkinter for when a parent is not explicitly given. There is no default root before the user creates a *Tk* class.

CHAPTER

FIVE

TK LICENSE ATTRIBUTION

Since these docs are based very closely to the Tk man pages, it is required that a copy of the license notice is included:

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, ActiveState Corporation, Apple Inc. and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

- genindex

Note: Not all options detailed here will be available on all Tk versions.

Note: Internal tkinter methods are not documented here as they are subject to change without notice.

Note: To maintain consistency with Python's built-in `help` function, this guide tries to maintain the tkinter method signatures. As-such, they will not always match those found on the Tk man page.

Note: This reference makes no claim about how up-to-date or accurate the content is. It was up-to-date at the time of writing and I kindly ask any issues are reported on the Github page (trivial patches are also welcome!).

Note: This work is licensed under CC0, but as it is heavy derived from the Tk man pages, any derivatives of this work must also include the Tk attribution.

INDEX

A

addtag_above() (*Canvas method*), 14
addtag_all() (*Canvas method*), 14
addtag_below() (*Canvas method*), 14
addtag_closest() (*Canvas method*), 14
addtag_enclosed() (*Canvas method*), 14
addtag_overlapping() (*Canvas method*), 15
addtag_withtag() (*Canvas method*), 15
after() (*Widget method*), 5
after_cancel() (*Widget method*), 5
after_idle() (*Widget method*), 5
anchor() (*Widget method*), 5

B

bbox() (*Canvas method*), 15
bell() (*Widget method*), 5
bind() (*Widget method*), 5
bind_all() (*Widget method*), 5
bind_class() (*Widget method*), 5
bindtags() (*Widget method*), 5
BitmapImage (*built-in class*), 10

C

Canvas (*built-in class*), 10
canvasx() (*Canvas method*), 15
canvasy() (*Canvas method*), 15
cget() (*Widget method*), 5
clipboard_append() (*Widget method*), 5
clipboard_clear() (*Widget method*), 5
clipboard_get() (*Widget method*), 5
columnconfigure() (*Widget method*), 5
config() (*Widget method*), 5
configure() (*Widget method*), 5
coords() (*Canvas method*), 15
create_arc() (*Canvas method*), 15
create_bitmap() (*Canvas method*), 18
create_image() (*Canvas method*), 19
create_line() (*Canvas method*), 19
create_oval() (*Canvas method*), 22
create_polygon() (*Canvas method*), 24
create_rectangle() (*Canvas method*), 27
create_text() (*Canvas method*), 29
create_window() (*Canvas method*), 31

D

dchars() (*Canvas method*), 32
default root, **41**
delete() (*Canvas method*), 32
deletecommand() (*Widget method*), 5
destroy() (*Widget method*), 5
dtag() (*Canvas method*), 32

E

event_add() (*Widget method*), 5
event_delete() (*Widget method*), 5
event_generate() (*Widget method*), 5
event_info() (*Widget method*), 5

F

find_above() (*Canvas method*), 32
find_all() (*Canvas method*), 32
find_below() (*Canvas method*), 32
find_closest() (*Canvas method*), 32
find_enclosed() (*Canvas method*), 32
find_overlapping() (*Canvas method*), 33
find_withtag() (*Canvas method*), 33
focus() (*Canvas method*), 33
focus_displayof() (*Widget method*), 5
focus_force() (*Widget method*), 6
focus_get() (*Widget method*), 6
focus_lastfor() (*Widget method*), 6
focus_set() (*Widget method*), 6
forget() (*Widget method*), 6

G

getboolean() (*Widget method*), 6
getdouble() (*Widget method*), 6
getint() (*Widget method*), 6
gettags() (*Canvas method*), 33
getvar() (*Widget method*), 6
grab_current() (*Widget method*), 6
grab_release() (*Widget method*), 6
grab_set() (*Widget method*), 6
grab_set_global() (*Widget method*), 6

grab_status() (*Widget method*), 6
grid() (*Widget method*), 6
grid_anchor() (*Widget method*), 6
grid_bbox() (*Widget method*), 6
grid_columnconfigure() (*Widget method*), 6
grid_configure() (*Widget method*), 6
grid_forget() (*Widget method*), 6
grid_info() (*Widget method*), 6
grid_location() (*Widget method*), 6
grid_propagate() (*Widget method*), 6
grid_remove() (*Widget method*), 6
grid_rowconfigure() (*Widget method*), 6
grid_size() (*Widget method*), 6
grid_slaves() (*Widget method*), 6

I

icursor() (*Canvas method*), 33
image_names() (*Widget method*), 6
image_types() (*Widget method*), 6
index() (*Canvas method*), 33
info() (*Widget method*), 6
insert() (*Canvas method*), 33
itemcget() (*Canvas method*), 34
itemconfig() (*Canvas method*), 34
itemconfigure() (*Canvas method*), 34

K

keys() (*Widget method*), 6

L

lift() (*Canvas method*), 34
location() (*Widget method*), 6
lower() (*Canvas method*), 34

M

mainloop() (*Widget method*), 6
move() (*Canvas method*), 34
moveto() (*Canvas method*), 34

N

nametowidget() (*Widget method*), 7

O

option_add() (*Widget method*), 7
option_clear() (*Widget method*), 7
option_get() (*Widget method*), 7
option_readfile() (*Widget method*), 7

P

pack() (*Widget method*), 7
pack_configure() (*Widget method*), 7
pack_forget() (*Widget method*), 7
pack_info() (*Widget method*), 7
pack_propagate() (*Widget method*), 7
pack_slaves() (*Widget method*), 7
PhotoImage (*built-in class*), 35
place() (*Widget method*), 7
place_configure() (*Widget method*), 7
place_forget() (*Widget method*), 7
place_info() (*Widget method*), 7
place_slaves() (*Widget method*), 7
postscript() (*Canvas method*), 34
propagate() (*Widget method*), 7

Q

quit() (*Widget method*), 7

R

register() (*Widget method*), 7
rowconfigure() (*Widget method*), 7

S

scale() (*Canvas method*), 34
scan_dragto() (*Canvas method*), 34
scan_mark() (*Canvas method*), 34
Scrollbar (*built-in class*), 35
select_adjust() (*Canvas method*), 34
select_clear() (*Canvas method*), 34
select_from() (*Canvas method*), 34
select_item() (*Canvas method*), 34
select_to() (*Canvas method*), 34
selection_clear() (*Widget method*), 7
selection_get() (*Widget method*), 7
selection_handle() (*Widget method*), 7
selection_own() (*Widget method*), 7
selection_own_get() (*Widget method*), 7
send() (*Widget method*), 7
set() (*Scrollbar method*), 35
setvar() (*Widget method*), 7
size() (*Widget method*), 7
slaves() (*Widget method*), 7

T

tag_bind() (*Canvas method*), 34
tag_lower() (*Canvas method*), 34
tag_raise() (*Canvas method*), 34
tag_unbind() (*Canvas method*), 34
Tk (*built-in class*), 3
tk_bisque() (*Widget method*), 7
tk_focusFollowsMouse() (*Widget method*), 7
tk_focusNext() (*Widget method*), 7
tk_focusPrev() (*Widget method*), 8
tk_setPalette() (*Widget method*), 8
tk_strictMotif() (*Widget method*), 8
tkraise() (*Canvas method*), 34
Toplevel (*built-in class*), 35

type() (*Canvas method*), 34

U

unbind() (*Widget method*), 8
unbind_all() (*Widget method*), 8
unbind_class() (*Widget method*), 8
update() (*Widget method*), 8
update_idletasks() (*Widget method*), 8

W

wait_variable() (*Widget method*), 8
wait_visibility() (*Widget method*), 8
wait_window() (*Widget method*), 8
waitvar() (*Widget method*), 8
Widget (*built-in class*), 5
winfo_atom() (*Widget method*), 8
winfo_atomname() (*Widget method*), 8
winfo_cells() (*Widget method*), 8
winfo_children() (*Widget method*), 8
winfo_class() (*Widget method*), 8
winfo_colormapfull() (*Widget method*), 8
winfo_containing() (*Widget method*), 8
winfo_depth() (*Widget method*), 8
winfo_exists() (*Widget method*), 8
winfo_fpixels() (*Widget method*), 8
winfo_geometry() (*Widget method*), 8
winfo_height() (*Widget method*), 8
winfo_id() (*Widget method*), 8
winfo_interps() (*Widget method*), 8
winfo_ismapped() (*Widget method*), 8
winfo_manager() (*Widget method*), 8
winfo_name() (*Widget method*), 8
winfo_parent() (*Widget method*), 8
winfo_pathname() (*Widget method*), 8
winfo_pixels() (*Widget method*), 8
winfo_pointerx() (*Widget method*), 8
winfo_pointerxy() (*Widget method*), 8
winfo_pointery() (*Widget method*), 8
winfo_reqheight() (*Widget method*), 9
winfo_reqwidth() (*Widget method*), 9
winfo_rgb() (*Widget method*), 9
winfo_rootx() (*Widget method*), 9
winfo_rooty() (*Widget method*), 9
winfo_screen() (*Widget method*), 9
winfo_screencells() (*Widget method*), 9
winfo_screendepth() (*Widget method*), 9
winfo_screenheight() (*Widget method*), 9
winfo_screenmmheight() (*Widget method*), 9
winfo_screenmmwidth() (*Widget method*), 9
winfo_screenvisual() (*Widget method*), 9
winfo_screenwidth() (*Widget method*), 9
winfo_server() (*Widget method*), 9
winfo_toplevel() (*Widget method*), 9
winfo_viewable() (*Widget method*), 9
winfo_visual() (*Widget method*), 9
winfo_visualid() (*Widget method*), 9
winfo_visualsavailable() (*Widget method*), 9
winfo_vrootheight() (*Widget method*), 9
winfo_vrootwidth() (*Widget method*), 9
winfo_vrootx() (*Widget method*), 9
winfo_vrooty() (*Widget method*), 9
winfo_width() (*Widget method*), 9
winfo_x() (*Widget method*), 9
winfo_y() (*Widget method*), 9

X

XView (*built-in class*), 9
xview() (*XView method*), 9
xview_moveto() (*XView method*), 9
xview_scroll() (*XView method*), 9

Y

YView (*built-in class*), 9
yview() (*YView method*), 9
yview_moveto() (*YView method*), 9
yview_scroll() (*YView method*), 9